CMake 入门与工程实践

2026/11/8 WHUAI

COURSE MAP

本次课程内容

01

编译原理与工具

实验：Hello World

`01_Compiler`

02

CMake 基础语法

实验：交互式计算器

`02_Calculator`

03

静态库与动态库

实验：学生成绩系统

`03_Libraries`

04

真实工程与第三方库

实验：OpenCV 作业

`04_OpenCV_demo`

PART 01 · CONCEPT

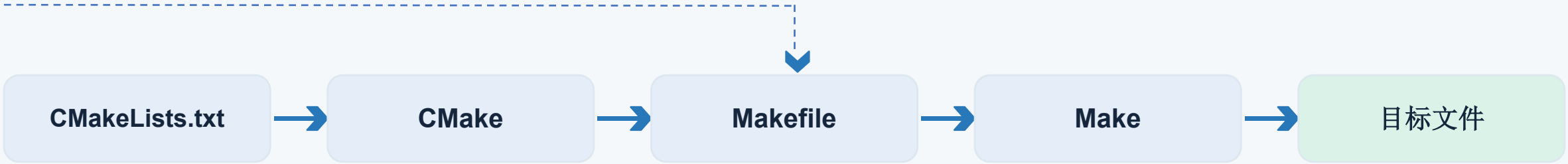
代码如何变成可执行程序？




```bash
g++ -E hello.cpp -o hello.i
g++ -S hello.cpp -o hello.s
g++ -c hello.cpp -o hello.o
g++ hello.o -o hello
```

PART 01 · CONCEPT

CMake是做什么的？



CMake 是一种跨平台的项目构建工具，主要用于管理和组织 C/C++ 工程的编译过程。通常情况下，我们可以通过编写 Makefile 来指定源文件的编译规则、文件之间的依赖关系以及最终可执行程序的生成方式，再通过 make 命令完成项目构建。然而，Makefile 往往与具体的操作系统和编译环境有关，当工程规模较大、源文件较多时，手动编写和维护 Makefile 不仅工作量大，还容易出现依赖关系配置错误的问题。

CMake 的出现就是为了简化这一过程。 开发者只需要编写一个 CMakeLists.txt 文件，描述工程包含哪些源文件、使用什么编译器、需要链接哪些库，以及最终生成什么程序。CMake 就能够根据当前平台和指定的构建工具，自动生成对应的 Makefile等构建文件。随后，再由构建工具调用编译器完成实际编译。

工具安装与版本检查

```bash
sudo apt update
sudo apt install -y build-essential cmake vim tree pkg-config

gcc --version     g++ --version
make --version    cmake --version
which gcc        which cmake
```

工具	主要任务	检查命令
gcc / g++	实际编译与链接	gcc --version
make	执行 Makefile 构建规则	make --version
cmake	配置、生成和管理构建系统	cmake --version

实操 1：只用 GCC / G++ 编译 Hello World

bash

```
gcc hello.c -o hello_c
./hello_c
```

bash

```
g++ -std=c++17 hello.cpp -o hello_cpp
./hello_cpp
```

CMake 项目结构

tree

```
02_Calculator/
├── CMakeLists.txt
├── include/calculator.h
├── src/main.cpp
├── src/add.cpp  sub.cpp  mul.cpp  div.cpp
└── build/          # 构建目录
```

bash

```
cd ~/CMake_Practice/02_Calculator/build
cmake ..                    # 配置并生成 Makefile
make                        # 编译链接
../bin/calculator            # 运行程序
```

CMakeLists.txt 用于编写构建规则，include 存放头文件，src 存放源代码，build 则专门存放编译过程中生成的文件。统一进入 build 目录执行 cmake .. 和 make，这样能够将源代码与构建产物分开，方便管理。

CMake 基础语法 A：版本、工程、变量

命令 / 语法	参数说明	用途
cmake_minimum_required(VERSION 3.16)	VERSION = 需要的最低 CMake 版本	约束版本 / 策略
project(Calculator LANGUAGES CXX)	项目名；LANGUAGES 指定语言，等	定义工程与语言
set(SRC_DIR ${PROJECT_SOURCE_DIR}/src)	变量名 + 值；${} 读取变量	定义源目录等变量
message(STATUS "...")	STATUS = 普通状态输出	打印配置诊断

CMake 基础语法 B：搜索源、头文件与创建目标

```cmake
set(SRC_DIR ${PROJECT_SOURCE_DIR}/src)
file(GLOB SRC_FILES ${SRC_DIR}/*.cpp)
add_executable(calculator ${SRC_FILES})

target_include_directories(calculator PUBLIC
    "${PROJECT_SOURCE_DIR}/include")
```

命令	关键参数
file(GLOB OUT ... pattern)	OUT：结果变量；pattern：通配符
aux_source_directory(DIR OUT)	DIR：搜索目录；OUT：结果变量
add_executable(NAME sources...)	NAME：目标名；sources：源文件列表
include_directories(DIR)/target_include_directories(NAME SCOPE DIR)	NAME：目标名；DIR：搜索目录

当工程包含多个源文件时，我们可以用 file(GLOB ...) 或 aux_source_directory() 搜索文件，并将结果保存到变量中。随后通过 add_executable() 指定生成的可执行程序和参与编译的源文件。
GLOB: 将指定目录下搜索到的满足条件的所有文件名生成一个列表，并将其存储到变量中。

实操 2：交互式加减乘除计算器

cmake

```
cmake_minimum_required(VERSION 3.10)

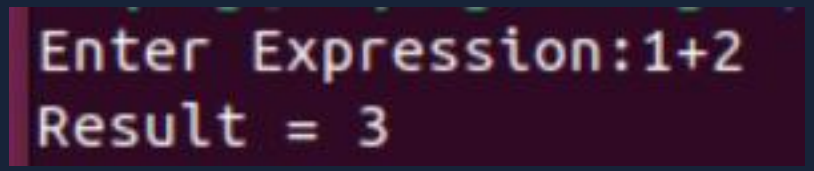
project(calculator)

set(SRC_DIR ${PROJECT_SOURCE_DIR}/src)
set(INC_DIR ${PROJECT_SOURCE_DIR}/include)
set(EXECUTABLE_OUTPUT_PATH ${PROJECT_SOURCE_DIR}/bin)

include_directories(${INC_DIR})

file(GLOB SRC_FILES ${SRC_DIR}/*.cpp)
add_executable(calculator ${SRC_FILES})

#target_include_directories(calculator PUBLIC ${INC_DIR})
```



目标：把加、减、乘、除分别写在不同的源文件中，并通过头文件统一声明函数。随后使用 CMake 搜索这些源文件、设置头文件路径并生成计算器程序。

创建与链接静态 / 动态库

静态库在 Linux 中通常以 .a 为后缀，例如 libstats.a。当主程序链接静态库时，链接器会将程序需要的目标代码合并到最终的可执行文件中。可以把它理解成：在生成程序时，把需要的功能代码直接装进程序内部。
因此，程序编译链接完成后，通常不再需要原来的 .a 文件就可以运行相关功能。它的优点是部署相对简单，不需要额外管理这个静态库文件；缺点是如果多个程序都使用同一个静态库，各程序通常都需要保存自己的一份相关代码，而且修改库之后通常需要重新链接程序。
在 CMake 中，使用 STATIC 创建静态库：
add_library(stats STATIC stats.cpp)

动态库在 Linux 中通常以 .so 为后缀，例如 libreport.so。与静态库不同，动态库不会在链接时整体复制到可执行文件中，而是作为独立的库文件存在。程序启动时，由操作系统的动态链接器寻找并加载所需的动态库。可以把它理解成：程序自己不携带这部分功能的完整代码，而是在运行时找到对应的功能模块并使用。
这样做的好处是，多个程序可以共享同一个动态库，通常能够减少重复代码占用的空间；在保持兼容性的情况下，也可以独立更新动态库。但是程序运行时必须能够找到相应的 .so 文件，否则可能出现动态库加载失败的问题。
在 CMake 中，使用 SHARED 创建动态库：
add_library(report SHARED report.cpp)

创建与链接静态 / 动态库

```cmake
add_library(stats STATIC src/stats.cpp)
add_library(report SHARED src/report.cpp)

target_include_directories(stats PUBLIC ${PROJECT_SOURCE_DIR}/include)
target_include_directories(report PUBLIC ${PROJECT_SOURCE_DIR}/include)

add_executable(score_system main.cpp)
target_link_libraries(score_system PUBLIC stats report)
```

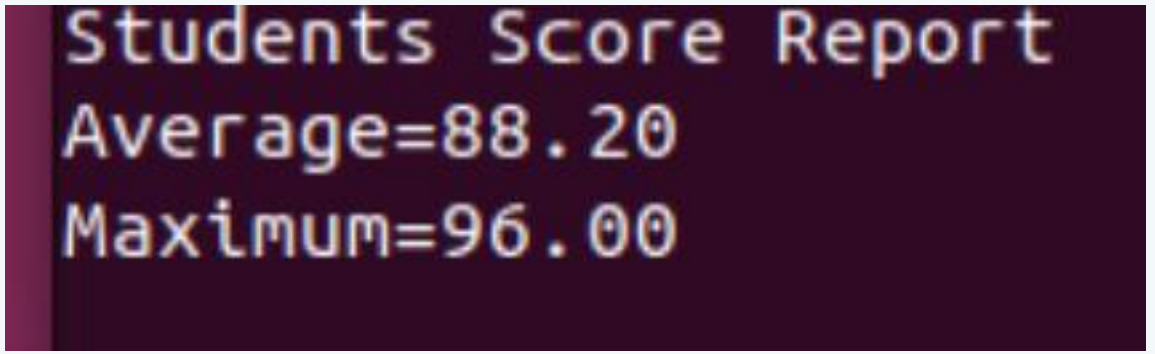
语法	重要参数
add_library(NAME STATIC/SHARED source...)	目标名 / 库类型 / 源文件
target_link_libraries(TARGET SCOPE lib...)	目标 / 传播范围 / 依赖的库

PUBLIC：后面的库会被Link到前面的target中，当前目标使用，也向依赖它的目标传播。
PRIVATE：在private后面的库被link到前面的target中，当前目标自己使用，不作为公共使用要求传播。
INTERFACE：在interface后面引入的库不会被链接到前面的target中。

实操 3：学生成绩统计系统

```
tree

03_Libraries/
├── CMakeLists.txt
├── include/stats.h  report.h
├── src/stats.cpp  report.cpp
├── main.cpp
└── build
├── bin
└── lib
```



要求：将平均分和最高分的计算封装成静态库 libstats.a，将报告格式化功能封装成动态库 libreport.so，最后由主程序统一调用。

实操 3：学生成绩统计系统

```
tree
```

```
cmake_minimum_required(VERSION 3.10)

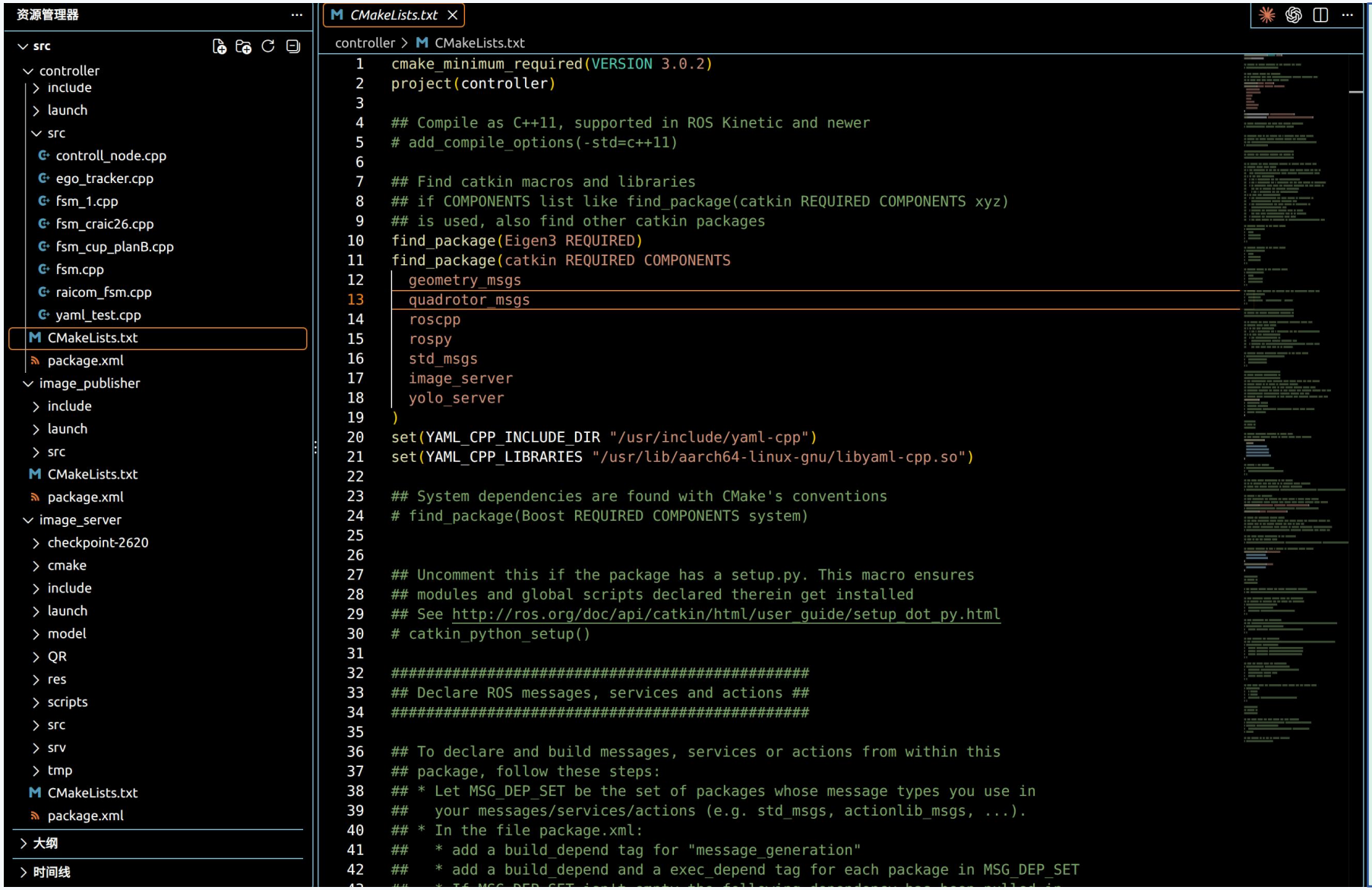
project(score_system)

set(SRC_DIR ${PROJECT_SOURCE_DIR}/src)
set(INC_DIR ${PROJECT_SOURCE_DIR}/include)
set(EXECUTABLE_OUTPUT_PATH ${PROJECT_SOURCE_DIR}/bin)
set(LIBRARY_OUTPUT_PATH ${PROJECT_SOURCE_DIR}/lib)

include_directories(${INC_DIR})

add_library(stats STATIC ${SRC_DIR}/stats.cpp)
add_library(report SHARED ${SRC_DIR}/report.cpp)

add_executable(score_system main.cpp)
target_link_libraries(score_system stats report)
```



如果项目很大，或者项目中有很多的源码目录，在通过CMake管理项目的时候如果只使用一个CMakeLists.txt，那么这个文件相对会比较复杂，有一种化繁为简的方式就是给每个源码目录都添加一个CMakeLists.txt文件（头文件目录不需要），这样每个文件都不会太复杂，而且更灵活，更容易维护。

add_subdirectory(source_dir [binary_dir] [EXCLUDE_FROM_ALL])

第三方库

```bash
sudo apt update
sudo apt install -y libopencv-dev pkg-config
pkg-config --modversion opencv4
dpkg -L libopencv-dev | grep OpenCVConfig.cmake
```

实际工程中，我们往往需要使用已经开发好的第三方库，例如 OpenCV。首先通过 Ubuntu 的软件包管理器安装 OpenCV 开发包，然后使用 find_package() 让 CMake 查找其安装配置。

```cmake
find_package(OpenCV REQUIRED)

target_include_directories(graphics PUBLIC ${OpenCV_INCLUDE_DIRS})
target_link_libraries(graphics PUBLIC ${OpenCV_LIBS})
```

ASSIGNMENT

课后作业

01 **复现工程**

自己完成课上01-03三个cmake实验，验证结果

02 **完成测试**

编写04实验的顶层CMakeLists.txt，完成工程构建并得到结果

03 自行拓展

修改输出的图片内容，可以修改原代码添加新的形状，也可以新建新模块并编写代码和CMakeLists.txt

04 **提交结果**

将补全的完整四个实验打包发送。

